PRACTICAL WEEK 3

Task 1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

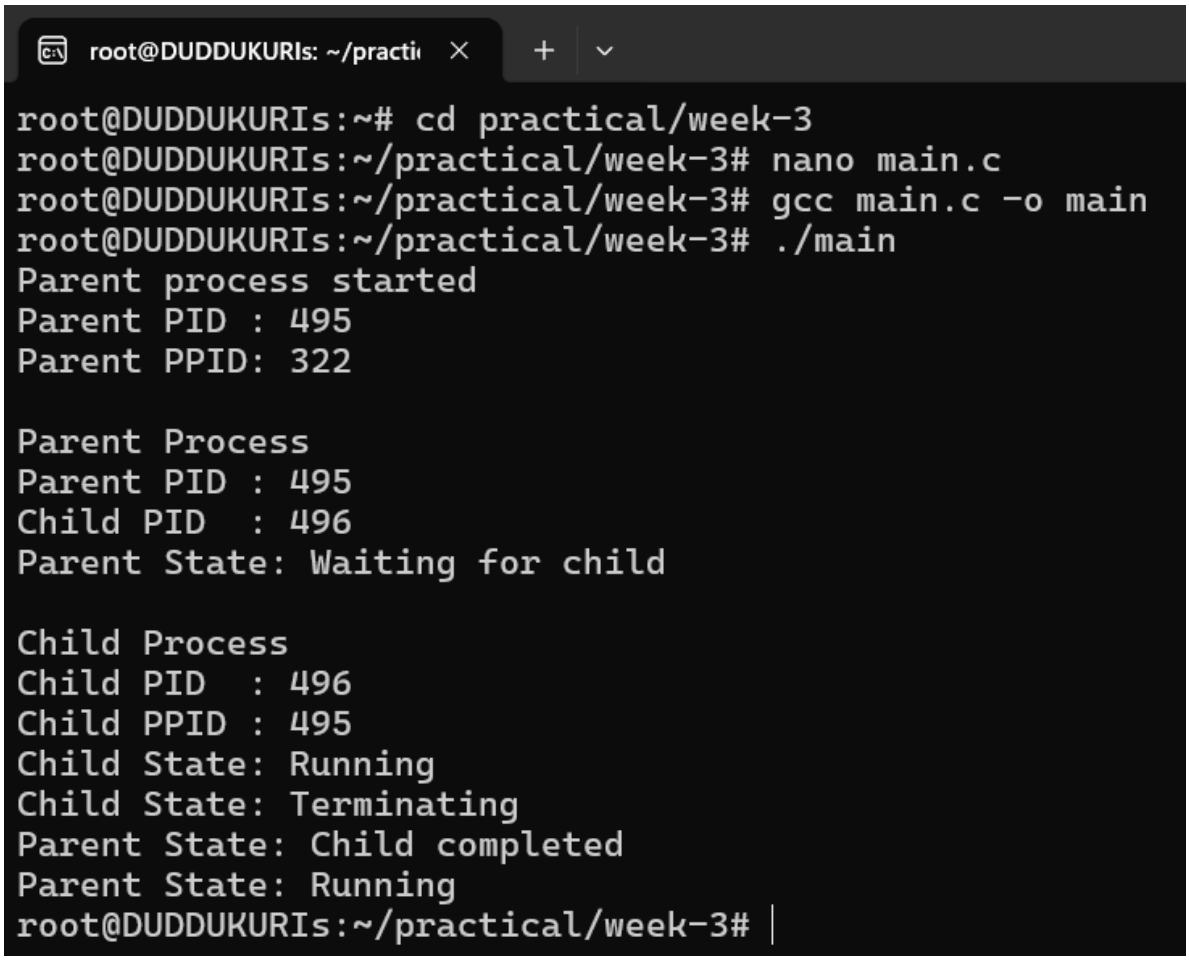
Source code:

root@DUDDUKURIs: ~/practi

GNU nano 7.2 main.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID  : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child State: Running\n");

        sleep(2);

        printf("Child State: Terminating\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        printf("Parent State: Waiting for child\n");

        wait(NULL);

        printf("Parent State: Child completed\n");
        printf("Parent State: Running\n");
    }

    return 0;
}
```

1) To save press control O
2) To come back to shell press control X
3) Then DO compilation



Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
root@DUDDUKURIs:~/practical/week-3# nano main2.c
root@DUDDUKURIs:~/practical/week-3# gcc main2.c -o main2
```

root@DUDDUKURIs: ~/practi

GNU nano 7.2 main2.c *

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process started\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    printf("\nProcess State: Running\n");

    printf("Process State: Sleeping/Waiting\n");
    sleep(10);

    printf("Process State: Running again\n");

    printf("Process State: Terminating\n");

    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs:~/practical/week-3# nano main2.c
root@DUDDUKURIs:~/practical/week-3# gcc main2.c -o main2
root@DUDDUKURIs:~/practical/week-3# ./main2
Process started
PID : 562
PPID : 322

Process State: Running
Process State: Sleeping/Waiting
Process State: Running again
Process State: Terminating
root@DUDDUKURIs:~/practical/week-3#
```